

Exam3StudyGuideGPT

W4111 Introduction to Databases, Spring 2026. Open-book study sheet for Exam 3.

Scope used for this guide: normalization from Lecture 6; all in-scope material from Lectures 9, 10, 11, and the Lecture 12 REST/web applications deck; the sample questions are treated as non-exhaustive examples of question style.

Source slide decks:

- Lecture 6:
Lectures/Lecture_06/W4111-2026-01-Lecture-6-ER-Relational-SQL-5-REST-Normalization-v1.pdf
- Lecture 9: Lectures/Lecture_09/W4111-2026-01-Lecture-09-Module-II-NoSQL-3-v1.pdf
- Lecture 10: Lectures/Lecture_10/W4111-2026-01-Lecture-10-Module-II-4-v1.pdf
- Lecture 11: Lectures/Lecture_11/W4111-2026-01-Lecture-11-Module-II-4-v1.pdf
- Lecture 12: Lectures/Lecture_12/Lecture-12-Module IV, REST, web applications-V2.pdf

Table of Contents

- Exam Answer Strategy and Topic Map - source: sample questions plus all listed decks; guide PDF page 2
- Normalization, Functional Dependencies, and Keys - source: Lecture 6 slides 49-76; guide PDF page 3
- Query Processing, Selection, and Join Algorithms - source: Lecture 9 slides 11-52; guide PDF page 5
- Query Optimization and Evaluation Plans - source: Lecture 9 slides 53-78; Lecture 10 slides 8-33; guide PDF page 7
- NoSQL, MongoDB, Neo4j, and Graph Thinking - source: Lecture 9 slides 79-101; guide PDF page 9
- Transactions, ACID, Logging, and Recovery - source: Lecture 10 slides 34-56; Lecture 11 slides 10-16; guide PDF page 11
- Serializability, Schedules, and Isolation Levels - source: Lecture 10 slides 57-69; Lecture 11 slides 17-44; guide PDF page 13
- Locking, Strict 2PL, and Deadlocks - source: Lecture 11 slides 37-44; Lecture 12 slides 7-20; guide PDF page 15
- ACID vs BASE, CAP, Scalability, and Replication - source: Lecture 10 slides 70-79; Lecture 11 slides 45-54; guide PDF page 16
- Big Data, Data Warehousing, Star Schema, OLAP, ETL/ELT, MapReduce, Spark - source: Lecture 11 slides 55-77; Lecture 12 slides 21-56 and 69-71; guide PDF page 18
- REST and Resource-Oriented Design - source: Lecture 9 slides 102-113; Lecture 10 slides 80-92; Lecture 11 slides 78-90; Lecture 12 slides 57-67 and 73-75; guide PDF page 20

Exam Answer Strategy and Topic Map

Source: exam-3-sample-questions.md plus all in-scope lecture decks. Guide PDF page 2.

Use this sheet as a lookup map. For written answers, give the definition, the rule/test, and the consequence. For data examples, cite specific tuple pairs or schedule conflicts.

High-probability reasoning patterns:

- Functional dependency from data: $X \rightarrow Y$ holds in the shown instance if every pair of tuples with the same X values also has the same Y values. It fails if one counterexample pair has same X and different Y .
- Candidate key: compute attribute closure. K is a candidate key if K^+ contains all attributes and no proper subset of K is also a superkey.
- Normal form answer: list candidate keys, identify prime attributes, test each non-trivial FD against BCNF/3NF rules, then decompose if needed.
- Query plan answer: distinguish logical equivalence from physical implementation. The best local operator can be bad globally if it destroys useful ordering or prevents pipelining.
- Join answer: state preconditions first: index available, equi-join/natural join, sorted inputs, relation sizes, memory.
- Transaction answer: identify which ACID property is at risk, then name the DBMS mechanism: logging/WAL/recovery, locks/2PL, isolation level, or rollback.
- Concurrency answer: define the schedule property, then explain with conflicts, precedence graph cycles, lock rules, or wait-for graph cycles.
- Distributed/scaling answer: name the tradeoff. Scale-out and high availability often weaken immediate consistency or make joins/constraints harder.
- REST answer: model resources and collections first, then map HTTP methods to CRUD.
- Data warehouse answer: fact table stores measurements/events; dimension tables describe analysis axes; OLAP operations aggregate/slice/pivot/drill.

Mini-map from sample questions to guide pages:

- Q1-Q2: FDs, keys, 3NF/BCNF - page 3.
- Q3-Q8: join algorithms, indexes, materialization, plans, conjunction/disjunction - pages 5-7.
- Q9-Q16, Q19: transactions, isolation, serializability, rollback, 2PL, deadlocks - pages 11-15.
- Q11, Q13: BASE, CAP, scale-up/scale-out, shared nothing - page 16.
- Q17: REST resources/endpoints - page 20.
- Q18, Q20: ETL/ELT, data engineering, star schema - page 18.

Open-book tactic: if a question says "briefly justify," include the exact counterexample, conflict edge, or normal-form rule. Definitions alone usually leave points on the table.

Normalization, Functional Dependencies, and Keys

Source: Lecture 6 slides 49-76. Guide PDF page 3.

Why normalization exists:

- Bad wide schemas repeat facts and create update anomalies. Example: if dept_name $\rightarrow$ building, budget, storing department facts in every faculty row means one bad update can make one department appear to be in two buildings.
- Normalization asks whether a relation is in "good" form. If not, decompose into smaller relations that are good, lossless, and preferably dependency preserving.
- Denormalization can be chosen for performance, but it trades faster reads for extra storage, update work, and possible inconsistency. Materialized views are a managed denormalization pattern.

Functional dependencies:

- $\alpha \rightarrow \beta$ means any two legal tuples that agree on all attributes in α must agree on all attributes in β .
- An FD is trivial if β is a subset of α .
- A shown instance can accidentally satisfy an FD that does not hold for all legal instances. For exam questions that say "based only on data shown," use only the tuple evidence.
- Closure F^+ is every FD logically implied by F . Attribute closure X^+ is every attribute determined by X under F .

Keys:

- Superkey: $K \rightarrow R$, meaning K^+ contains all attributes of relation R .
- Candidate key: a minimal superkey. Remove any attribute from K , and it should no longer determine all attributes.
- Prime attribute: an attribute contained in at least one candidate key.

Fast closure algorithm:

1. Start with $X^+ = X$.
2. Repeatedly apply any FD $A \rightarrow B$ where A is contained in X^+ ; add B to X^+ .
3. Stop when no more attributes can be added.
4. If X^+ contains every attribute, X is a superkey. Check minimality for candidate key.

Normal forms:

- 1NF: values are atomic; no repeating groups or multi-valued cells.
- 2NF: 1NF plus no partial dependency of a non-key attribute on part of a composite key.
- 3NF: for every non-trivial $\alpha \rightarrow \beta$ in F^+ , either α is a superkey, or each attribute in β - α is prime.
- BCNF: for every non-trivial $\alpha \rightarrow \beta$ in F^+ , α must be a superkey. BCNF is stricter than 3NF.
- If a relation is in BCNF, it is in 3NF. A relation can be in 3NF but not BCNF when the dependent attribute is prime.

Decomposition:

- Lossless decomposition: joining decomposed relations recreates the original relation without spurious tuples.
- Dependency preserving: constraints can be enforced by checking the decomposed relations independently.
- BCNF decomposition for violating $\alpha \rightarrow \beta$: replace R with $(\alpha \cup \beta)$ and $(R - (\beta - \alpha))$.

- BCNF may sacrifice dependency preservation. 3NF can always preserve losslessness and dependency preservation, but may allow some redundancy.

Exam template for 3NF vs BCNF:

- Find candidate keys.
- Mark prime attributes.
- For each FD, ask: is left side a superkey? If yes, OK for both 3NF and BCNF.
- If not, BCNF fails. 3NF passes only if every right-side attribute not already on left is prime.
- If decomposing to BCNF, use the violating FD and show resulting schemas.

Query Processing, Selection, and Join Algorithms

Source: Lecture 9 slides 11-52. Guide PDF page 5.

Query processing pipeline:

- Parser/translator checks syntax and converts SQL to a logical plan tree.
- Optimizer rewrites the logical plan and chooses physical algorithms.
- Engine executes the chosen evaluation plan, sometimes adapting based on indexes or runtime details.
- Basic steps: parsing/translation, optimization, evaluation.

Selection and index intuition:

- Full scan: check every block/tuple. Rough order: $O(N)$.
- B+ tree index lookup: traverse tree height, roughly $O(\log N)$, plus data fetches.
- Hash lookup: roughly $O(1)$ for equality, plus data fetches.
- Index scan helps only if the predicate uses the index search key.
- Secondary index on a non-key equality can be expensive if many matching records live on different blocks.
- Range predicates may or may not benefit from an index. If many records qualify, a scan can be cheaper than many random I/Os.

Conjunction vs disjunction:

- `name='Smith' AND dept_name='CS'` can use an index on name, fetch matching rows, then test `dept_name` in memory.
- `name='Smith' OR dept_name='CS'` cannot be solved by the name index alone if there is no index on `dept_name`; the engine still must find rows matching the unindexed side.
- For conjunctions, use the most selective useful index first. For disjunctions, index-union works only if all disjuncts have useful indexes; otherwise scan.

Join algorithms:

- Nested-loop join: for each tuple/block in outer relation, scan/probe the inner relation. Works for any join condition and needs no index, but can be very expensive.
- Block nested-loop join: improves nested loop by processing blocks; good when memory can hold useful chunks.
- Indexed nested-loop join: for each outer tuple, use an index on the inner join attribute. Best when join is equi/natural and the inner relation has a useful index. If both sides are indexed, make the relation with fewer tuples the outer.
- Merge join: sort both relations on join attributes, then merge. Best for equi/natural joins when inputs are already sorted or sorted output is useful later.
- Hash join: partition both relations by hash of join attributes, build an in-memory hash table on the build input, then probe. Best for large equi/natural joins when partitions fit memory.

Outer vs inner in nested-loop join:

- The outer relation drives how many times the inner relation is scanned/probed.
- With indexed nested-loop, choose the smaller relation as outer when the inner has an index; example: 1,000 rows in R, 1,000,000 rows in S, index on `S.join_attr` -> use indexed nested-loop with R outer and S inner.
- Without indexes, buffer behavior matters. If one relation fits in memory, use it where it avoids repeated I/O.

Other operations:

- Duplicate elimination: sorting or hashing groups duplicates together.
- Projection: project desired attributes, then eliminate duplicates if SQL semantics require it.
- Aggregation: sorting or hashing groups tuples; partial aggregation can maintain count/min/max/sum and combine later. For average, keep sum and count.

Materialization vs pipelining:

- Materialization stores intermediate results before parent operations read them. It is always applicable but writes/reads temporary data.
- Pipelining passes tuples directly from one operator to the next. It can avoid temporary disk I/O but not every operator can stream results.
- Sort and hash join often block pipelining because they need significant input before producing output.
- Demand-driven pipeline: parent calls next() on child iterators. Producer-driven pipeline: child pushes tuples into buffers.

Query Optimization and Evaluation Plans

Source: Lecture 9 slides 53-78; Lecture 10 slides 8-33. Guide PDF page 7.

Core idea:

- A logical query can have many equivalent relational algebra expressions.
- Each expression can have many physical implementations.
- An evaluation plan specifies the physical algorithm for each operation and how operations are coordinated.
- Cost-based optimization: generate equivalent expressions, annotate with physical plans, estimate cost from catalog statistics, choose the cheapest estimated plan.

Important statistics:

- Number of tuples and blocks.
- Tuple size.
- Number of distinct values for attributes.
- Selectivity of predicates.
- Estimated size of intermediate results.

Common rewrite rules:

- Split conjunctions: $\sigma(a \text{ AND } b)(E)$ can become $\sigma(a)(\sigma(b)(E))$.
- Selection is commutative: apply the more selective predicate first when useful.
- Consecutive projections can collapse to the final needed attribute list.
- Selection plus Cartesian product can become a theta join.
- Joins are commutative and associative for inner/natural joins, enabling join reordering.
- Push selections below joins when the predicate references only one side.
- Push projections below joins when you keep the needed output attributes plus join attributes.
- Union/intersection are commutative/associative; difference is not commutative.
- Outer joins do not obey all inner join rules. Be careful: outer joins are not generally associative, and some selection rewrites change null-preserving behavior.

Optimization heuristics:

- Push selections early to reduce relation sizes before joins.
- Push projections early to reduce tuple width, but keep join/grouping attributes needed later.
- Join smaller/selective intermediate results first.
- Choose access paths using indexes only when selectivity makes them cheaper than scans.
- Preserve useful order when a later merge join, grouping, or order-by can benefit.

Why local optimum can be globally bad:

- Hash join may be cheapest for one join, but merge join may output sorted data that saves a later sort.
- Nested-loop may be locally slower but allows pipelining.
- A very selective index lookup can be best for one predicate, but a different predicate may produce an intermediate result that makes the whole plan cheaper.

Logical vs physical example:

- $\pi_{\text{name}}(\sigma_{\text{salary} < 75000}(\text{instructor}))$ and $\sigma_{\text{salary} < 75000}(\pi_{\text{name, salary}}(\text{instructor}))$ are logically equivalent if salary is retained for the selection.
- Usually select before project/join when selection reduces rows. Usually project before join when projection reduces tuple width but keeps required columns.

EXPLAIN:

- DBMSs expose selected plans with EXPLAIN or related commands.
- PostgreSQL-style costs can show first-tuple cost and all-results cost.
- EXPLAIN ANALYZE includes actual runtime statistics, not only estimates.

Join-order dynamic programming:

- Best plan for a set of relations is built from best plans for subsets.
- For each split of relation set S into S1 and S-S1, compare join algorithms and keep the cheapest.
- Avoid enumerating every possible tree naively because equivalent-expression space is large.

NoSQL, MongoDB, Neo4j, and Graph Thinking

Source: Lecture 9 slides 79-101. Guide PDF page 9.

NoSQL classifications from the lecture:

- Relational is the foundational model.
- NoSQL families include document stores, key-value stores, column-family/wide-column stores, and graph databases.
- The point is not "NoSQL replaces SQL"; it is that different data/access patterns benefit from different models.

MongoDB concepts:

- RDBMS database -> MongoDB database.
- Table -> collection.
- Row/tuple -> document.
- Column -> field.
- Join -> often embedded documents, references, or aggregation pipeline operations such as \$lookup.
- Primary key -> _id by default.
- Server/tool/API examples: mongod, Compass, pymongo.

MongoDB operations:

- Create: insertOne(), insertMany(); in pymongo, names use snake case such as insert_one().
- Retrieve: find(), findOne() / find_one().
- Update: updateOne(), updateMany(), replaceOne().
- Delete: deleteOne(), deleteMany().
- find() generally takes a filter expression and a projection expression.
- find() returns a cursor/iterable, not just a single row.
- Projection on nested arrays can return more than expected; matching a document does not automatically trim nested array elements to only matching sub-elements.
- Aggregation pipeline operators mentioned include match, lookup, union, merge, sample, and related pipeline stages.
- MongoDB also has indexes, replication, sharding, and embedded MapReduce support.

MongoDB exam style:

- Filter: "which documents?" Projection: "which fields?"
- Use embedded documents when data is usually read/written together and bounded.
- Use references/lookup when data is large, shared, or independently updated.
- Remember that document databases can avoid some joins by storing related data together.

Neo4j / graph database thinking:

- Graph databases model nodes and relationships directly.
- Graph queries shine when the question is path-oriented: "who acted in which movies?", "which actors worked with both X and Y?", "how do you get from Kevin Bacon to Robert Longo?", or recommendation via shared relationships.
- In Cypher-like thinking, match node/relationship patterns, bind variables, filter, and return results.
- SQL can represent graphs with join tables, but multi-hop traversal can become join-heavy and hard to read.

Generic Cypher pattern memory aid:

- MATCH (a:Actor)-[:ACTED_IN]->(m:Movie) finds actor/movie paths.

- WHERE a.name = 'Tom Hanks' filters.
- RETURN m.title projects output.
- Variable-length paths use relationship repetitions in many graph query languages; check exact syntax from tutorial if needed.

Transactions, ACID, Logging, and Recovery

Source: Lecture 10 slides 34-56; Lecture 11 slides 10-16. Guide PDF page 11.

Transaction:

- A transaction is a unit of program execution that reads/writes data and should move the database from one consistent state to another.
- Example transfer: read A, subtract 50, write A, read B, add 50, write B, then commit or rollback.

ACID:

- Atomicity: all writes happen or none happen. If transfer fails after debiting A but before crediting B, money is lost unless partial effects are undone.
- Consistency: a correct transaction preserves integrity constraints and application invariants, such as total A+B in a transfer.
- Isolation: concurrent transactions should behave as if each transaction executes without interference from others.
- Durability: once commit is acknowledged, changes persist despite failures.

Transaction states:

- Active: transaction is executing.
- Partially committed: final statement executed, commit not fully durable yet.
- Failed: cannot continue normally.
- Aborted: rolled back; may restart or be killed.
- Committed: successful completion.

DBMS subsystems for ACID:

- Query processor schedules and executes operations.
- Buffer manager manages block/frame reads and writes.
- Log manager records transaction start, update, commit, abort, checkpoint, completion.
- Transaction manager coordinates scheduling, buffers, and logging.
- Recovery manager uses log records after failures.

Write-ahead logging:

- Before dirty data pages are written, the log record describing the change must be forced to stable storage.
- Redo: after crash, reapply committed changes that were logged but not reflected on disk.
- Undo: roll back uncommitted changes that reached disk.
- WAL makes durability and atomicity practical without forcing every data page to disk on every commit.

ARIES recovery:

- Analysis pass: identify transactions to undo and dirty pages; find redo start point.
- Redo pass: repeat history from redo LSN, skipping actions already reflected on pages.
- Undo pass: roll back incomplete transactions.

Force/steal intuition:

- Force every update to disk at commit: durable but slow.
- No-force: commit can return after log is durable, while data pages are written later.
- Steal: buffer manager may write uncommitted dirty pages to disk; needs undo.
- No-steal: avoids uncommitted data on disk but hurts memory/buffer performance.

Availability and replication:

- Active/passive: requests go to master; backup receives queued updates and takes over after failure.
- Active/active: multiple systems process requests; consistency needs coordination such as distributed commit.
- The more distributed the system, the more CAP/consistency tradeoffs matter.

Serializability, Schedules, and Isolation Levels

Source: Lecture 10 slides 57-69; Lecture 11 slides 17-44. Guide PDF page 13.

Why concurrency exists:

- Improves CPU/disk utilization and transaction throughput.
- Reduces average response time; short transactions need not wait behind long ones.
- Requires concurrency control to avoid destroying database consistency.

Schedules:

- A schedule is the chronological order of operations from concurrent transactions.
- It must include all operations and preserve each transaction's internal order.
- Serial schedule: transactions run one after another.
- Serializable schedule: concurrent schedule with the same effect as some serial schedule.
- Serializability is the main correctness criterion because if every transaction is correct alone, any serial-equivalent schedule is correct.

Conflict serializability:

- Conflicts happen when two operations from different transactions access the same item and at least one is a write.
- Read/read does not conflict. Read/write, write/read, and write/write conflict.
- A schedule is conflict serializable if non-conflicting swaps can transform it into a serial schedule.
- Precedence graph test: nodes are transactions; edge $T_i \rightarrow T_j$ if T_i has a conflicting operation before T_j on the same item.
- A schedule is conflict serializable iff the precedence graph is acyclic.
- If acyclic, a topological ordering gives a valid serial order.

View serializability:

- More general than conflict serializability.
- Preserves which transaction reads the initial value, which transaction's write each read sees, and final writes.
- Some schedules are view serializable but not conflict serializable. Conflict serializability is easier to test and commonly emphasized.

Recoverability and cascading rollback:

- Recoverable schedule: if T_j reads data written by T_i , then T_i commits before T_j commits.
- Cascading rollback: one abort forces other transactions to abort because they read uncommitted data from the failed transaction.
- Cascadeless schedule: if T_j reads data written by T_i , T_i commits before T_j reads it.
- Every cascadeless schedule is recoverable.

Isolation levels:

- Read uncommitted: can see uncommitted data. Dirty reads can occur.
- Read committed: reads only committed data, but repeated reads can return different committed values. Non-repeatable reads can occur.
- Repeatable read: repeated reads of the same record return same value; phantoms/range anomalies can still occur if range locks are not used.
- Serializable: equivalent to some serial execution; strongest standard level.

Common anomalies:

- Dirty read: read uncommitted data from another transaction.
- Non-repeatable read: same row read twice returns different values due to committed update by another transaction.
- Phantom read: rerunning a range query returns new/deleted rows due to another transaction.
- Lost update/write skew: two transactions read overlapping data and write decisions that violate intended constraints.

Cursor note:

- Isolation is often defined around what cursors have read/touched.
- REST cannot rely on server-side cursor conversation state in the same way because REST is stateless.

Locking, Strict 2PL, and Deadlocks

Source: Lecture 11 slides 37-44; Lecture 12 slides 7-20. Guide PDF page 15.

Lock modes:

- Shared lock S: needed before reading; multiple transactions can hold compatible shared locks.
- Exclusive lock X: needed before writing; incompatible with all other locks on the same item.
- A lock request proceeds only after the concurrency-control manager grants it.

Strict two-phase locking:

- Before reading, transaction gets an S lock.
- Before writing, transaction gets an X lock.
- All locks are released only when the transaction completes.
- If a transaction holds an X lock, no other transaction can get S or X on that object.
- Strict 2PL guarantees serializable schedules and simplifies aborts by preventing other transactions from reading uncommitted writes.

Non-strict 2PL:

- A transaction may release locks before commit, but after releasing any lock it cannot acquire more locks.
- Also guarantees serializability, but abort/recovery is harder because cascading aborts are possible.

Deadlock:

- A set of transactions is deadlocked if every transaction in the set waits for another transaction in the same set.
- Example pattern: T3 holds lock on B and wants A; T4 holds lock on A and wants B.
- To resolve, roll back at least one transaction and release its locks.

Deadlock prevention:

- Pre-declaration: require all locks before transaction begins.
- Ordered locking: impose partial order on data items and acquire locks only in that order.
- Wait-die: non-preemptive. Older transaction may wait for younger; younger transaction trying to wait for older is rolled back.
- Wound-wait: preemptive. Older transaction forces rollback of younger; younger waits for older.
- Timeout: wait only a fixed time, then rollback. Simple but can abort transactions unnecessarily and can cause starvation.
- Restart rolled-back transactions with original timestamp to avoid starvation in timestamp schemes.

Deadlock detection:

- Wait-for graph nodes are transactions.
- Edge $T_i \rightarrow T_j$ means T_i waits for a lock held incompatibly by T_j .
- Cycle in wait-for graph means deadlock.
- Detection runs periodically; then pick a victim.

Deadlock recovery:

- Victim selection tries to minimize cost.
- Total rollback aborts and restarts victim.
- Partial rollback rolls back only far enough to release needed locks.
- Avoid starvation by not repeatedly selecting the same or oldest transaction as victim.

ACID vs BASE, CAP, Scalability, and Replication

Source: Lecture 10 slides 70-79; Lecture 11 slides 45-54. Guide PDF page 16.

Eventual consistency:

- If no new updates are made, eventually all reads return the last updated value.
- Used to improve availability and scalability in distributed systems.
- The system converges after asynchronous propagation and conflict handling.
- Strong consistency across replicas requires coordination, locking, or distributed transactions, which becomes hard during failures/partitions.

BASE:

- Basically Available: reads/writes are available as much as possible, often across cluster nodes, without strong immediate consistency guarantees.
- Soft state: the system state may be changing or uncertain while replicas converge.
- Eventual consistency: after enough time without new writes, reads converge to expected state.
- BASE trades immediate consistency for availability, performance, and scale.

ACID vs BASE:

- ACID emphasizes correctness of transactional updates: all-or-nothing, consistent state transitions, isolation, durability.
- BASE emphasizes availability and convergence in distributed systems.
- Prefer ACID for money movement, inventory correctness, constraints, and operations where anomalies are unacceptable.
- Prefer BASE/eventual consistency for large-scale read-heavy systems, replicated web services, feeds, caches, or conflicts that can be reconciled later.

CAP theorem:

- Consistency: every read receives the most recent write or an error.
- Availability: every request receives a non-error response, without a guarantee that it is the latest value.
- Partition tolerance: system continues despite dropped or delayed network messages between nodes.
- In practical distributed systems, partitions/failures happen, so designs must choose how to balance consistency and availability.

Scale-up vs scale-out:

- Scale-up: replace machine with bigger CPU/memory/disk. Less incremental, disruptive, expensive at extreme size, and does not by itself improve availability.
- Scale-out: add machines. Incremental cost and supports availability via replication, but relational operations like joins, constraints, and referential integrity are harder across nodes.

Disk/data architectures:

- Shared disk: multiple DB servers share storage through NAS/SAN/RAID. Integrity requires distributed locking.
- Shared nothing: each node owns its data partition/shard. Router sends requests to shard based on partition function.
- Sharding: partition data by key; replication can improve availability but creates consistency tradeoffs.

Why joins are hard in scale-out:

- Join inputs may live on different shards.

- Moving data across network is expensive.
- Distributed locking/transactions are complex.
- Denormalization, embedding, precomputed views, or application-side reconciliation may be used instead.

Big Data, Data Warehousing, Star Schema, OLAP, ETL/ELT, MapReduce, Spark

Source: Lecture 11 slides 55-77; Lecture 12 slides 21-56 and 69-71. Guide PDF page 18.

Big data and data engineering:

- Data engineering is often most of the hard work before analytics/AI: gathering, cleaning, transforming, integrating, and loading analyzable data.
- Common analytics flow: gather data from sources, integrate into a common schema, generate aggregates/reports, analyze interactively with OLAP/statistics, build predictive models.
- 5 V's often associated with big data: volume, velocity, variety, veracity, value.

Data warehouse vs data lake:

- Data warehouse: integrated, curated, schema-oriented store for analysis and reporting.
- Data lake: stores raw or semi-structured data for later processing; often cheaper/flexible but needs governance.

ETL vs ELT:

- ETL: extract from sources, transform into target schema, then load into warehouse.
- ELT: extract, load raw-ish data first, then transform inside the target platform.
- ETL is useful when target requires clean structured data before loading. ELT is common when the target system has scalable processing.

Star schema:

- Fact table: records events/measurements, often numeric facts such as quantityOrdered, priceEach, revenue.
- Dimension tables: describe axes for analysis, such as date, location, product, customer.
- Star schema is useful because analytics asks aggregations by dimensions, such as sales by country, product line, and year.
- Lecture 12 Classicmodels example: facts are quantity ordered, price each, revenue; dimensions include location, date, and product.

OLAP:

- Online Analytical Processing: interactive analysis that summarizes and views data in different ways.
- Data cube: multidimensional generalization of a cross-tab.
- Slice: fix one dimension value.
- Dice: fix multiple dimension values or select a subcube.
- Roll-up: aggregate from fine detail to coarser level, such as day to month/year.
- Drill-down: move from coarse aggregate to finer detail.
- Pivot: change which dimensions appear as rows/columns.

MapReduce:

- Data-flow model with map and reduce operators running in parallel on partitions.
- Good for large-scale procedural processing, web logs, keyword indexes, PageRank-like tasks.
- Many computations are easier in SQL; MapReduce can be cumbersome for simple queries.
- Relational operations can be translated to MapReduce, and modern engines support joins/aggregations natively.

Blocks and parallelism:

- Normal file streams hide where records start/end and which blocks contain which records.
- Databases and big-data tools expose records, blocks, block locations, and partitions so work can run in

parallel and reduce I/O.

Spark / algebraic operations:

- Modern engines such as Spark support trees/graphs of algebraic operators.
- Relational model has a fixed closed set of operators over relations; Spark exposes RDDs/data frames and lets programmers define data-flow graphs and custom operators.
- RDD: resilient distributed dataset, a collection of records stored across machines.
- RDDs can be lazily computed from operations on other RDDs.

REST and Resource-Oriented Design

Source: Lecture 9 slides 102-113; Lecture 10 slides 80-92; Lecture 11 slides 78-90; Lecture 12 slides 57-67 and 73-75. Guide PDF page 20.

Web application stack:

- Full-stack web application commonly has browser UI, microservice/application server, and database.
- Web frameworks support web APIs, database access, routing, templating/session management, and code reuse.
- OpenAPI describes, produces, consumes, and visualizes machine-readable service APIs.
- FastAPI routers structure APIs into route groups; other frameworks use similar concepts.

REST:

- REST means Representational State Transfer.
- REST is an architectural style for stateless, reliable web-based applications.
- Important constraints: client-server separation, stateless requests, cacheable responses, uniform interface, layered system, optional code-on-demand.
- Stateless means each request contains enough information for the server to understand it; server should not rely on hidden conversation state.

Resources and collections:

- Resource: an abstract thing with state and possibly sub-resources.
- Collection resource: a list/container of same-type resources.
- Resource hierarchy can nest collections under resources, such as `/courses/{id}/sections`.
- A REST API is not necessarily a direct database exposure. Server logic maps abstract resources to files, DB rows, actions, or workflows.

Methods:

- POST: create a resource, often under a collection.
- GET: retrieve a resource or collection.
- PUT: update/replace a resource.
- DELETE: delete a resource.
- REST emphasizes many resources with a small fixed method set instead of many custom verbs like `openAccount()` or `deposit()`.

URLs, content types, and query parameters:

- Relative URL identifies the resource.
- Content types include `text/html` and `application/json`; Accept headers specify desired response type.
- Collection GET requests may use query parameters, e.g. `/customers?country=France&city=Paris`.
- Server translates query parameters into filtering logic, such as SQL WHERE predicates.

Endpoint design templates:

- Base collection: `/courses` supports GET, POST.
- Single resource: `/courses/{course_id}` supports GET, PUT, DELETE.
- Nested collection: `/courses/{course_id}/sections` retrieves/creates sections for a course.
- Nested collection for exam sample: `/sections/{section_id}/participants` retrieves participants in a section.
- Classicmodels-style examples from Lecture 12: `/customers`, `/customers/{customerNumber}`, `/customers/{customerNumber}/orders`, `/orders/{orderNumber}/orderdetails`.

Resource-oriented exam answer:

- First identify entity types and relationships.
- Use plural nouns for collections.
- Put contained resources under parent resources when navigation is natural.
- Use query parameters for filtering collections.
- Map methods to CRUD; do not invent verbs when an HTTP method plus resource path expresses the action.

Last-Minute Quick Checks

- FD question: search for counterexample pairs.
- Key question: compute closure, then prove minimality.
- 3NF/BCNF question: left side superkey? if not, are right-side attributes prime?
- Join question: equi-join? index? sorted? memory? relation sizes?
- Isolation question: dirty read, non-repeatable read, phantom, serializable.
- Deadlock question: wait-for graph cycle.
- Star schema question: facts are measurements; dimensions are analysis axes.
- REST question: resources are nouns; methods are CRUD.